

## \* Multithreading in Java $\Rightarrow$

Multithreading in Java allows concurrent execution of two or more parts of a program (called threads) for maximum utilization of CPU.

Each thread runs in parallel, improving performance in multi-core processors.

### ① Creating Threads in Java $\Rightarrow$

There are two ways to create a thread in Java.

① By extending Thread class.

② By Implementing Runnable Interface.

#### a) Extending Thread class $\Rightarrow$

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        for (int i=1; i<=5; i++) {
```

```
            System.out.println("Thread.currentThread().getName() + "-" + i);
```

```
        } try {
```

```
            Thread.sleep(500); //sleep for 500ms
```

```
        } catch (InterruptedException e) {
```

```
            System.out.println(e);
```

```
        }
```

preferred to use Runnable interface <sup>while</sup> to create thread because when we use Thread class to extend. Suppose we want to extend one more another class it will be not possible because in java multiple inheritance not allowed }

```
public class ThreadExample {
```

```
    public static void main (String[] args)
```

```
    {
        MyThread t1 = new MyThread();
```

```
        MyThread t2 = new MyThread();
```

```
        t1.start();
```

```
        t2.start();
```

```
    }
}
```

by Implementing Runnable Interface =>

```
class MyRunnable implements Runnable {
    public void run() {
```

```
        for (int i=1; i<=5; i++) {
```

```
            System.out.println(Thread.currentThread().
                                getName() + "-" + i);
```

```
        try {
```

```
            Thread.sleep(500);
```

```
        } catch (InterruptedException e) {
```

```
            System.out.println(e);
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```
}
```

```
public class RunnableExample {
```

```
    public static void main(String[] args) {
```

```
        Thread t1 = new Thread(new MyRunnable());
```

```
        Thread t2 = new Thread(new MyRunnable());
```

```
        t1.start();
```

```
        t2.start();
```

```
    }
```

```
}
```

## ② Thread Life Cycle :->

A thread in Java goes through several stages.

① **New** → Thread is created but not started (new Thread()).

start()

↓ ② **Runnable** → Thread is ready to run (start() is called).

③ **Running** → Thread is executing.

sleep(), wait(),

↓ ④ **Blocked / waiting** → Thread is waiting due to synchronization or sleep.

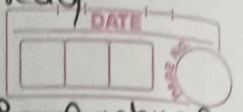
run() method ends, stop()

↓ ⑤ **Terminated** → Thread has finished execution.

Daemon thread → It is a service provider thread that provides services to the user thread.

(It is dependent on another thread)

(Best Example of Daemon thread is Garbage collector)



### ③ Thread Methods :-

Commonly used methods.

Method

Description

- ① start() → starts a new thread.
- ② run() → Entry point for thread execution.
- ③ sleep(ms) → Puts thread to sleep for a specified time.
- ④ join() → waits for a thread to finish.
- ⑤ isAlive → Checks if thread is still running.
- ⑥ setPriority(int) → Sets thread priority.
- ⑦ interrupt() → Interrupts a thread if it's sleeping or waiting.
- ⑧ setName(String name) and getName() → Set and get the name of the thread.
- ⑨ isDaemon() and setDaemon(true/false) → Checks or sets if a thread is daemon thread (background thread).

#### ④ Synchronization in Threads :->

Synchronization ensures that multiple threads do not access shared resources simultaneously, preventing race conditions.

Using Synchronized keyword :->

```
class SharedResource {
```

```
    synchronized void printNumbers(int n) {
```

```
        for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
```

```
        try {
            Thread.sleep(500);
```

```
        } catch (InterruptedException e) {
```

```
            System.out.println(e);
```

```
        }
```

```
    }
```

```
}
```

```
class MyThread extends Thread {
```

```
    SharedResource obj;
```

```
    MyThread(SharedResource obj) {
```

```
        this.obj = obj;
```

```
    }
```

```

public void run() {
    obj.printNumbers(5);
}
}
public class SyncExample {
    public static void main(String[] args) {
        SharedResource obj = new SharedResource();
        MyThread t1 = new MyThread(obj);
        MyThread t2 = new MyThread(obj);

        t1.start();
        t2.start();
    }
}

```

### ⑤ Thread Pooling :->

Thread pooling manages a pool of worker threads to execute tasks efficiently using Executor Framework.

Using ExecutorService.

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

```

class Task implements Runnable.

private int taskId;

public Task(int id) {  
    this.taskId = id;  
}

public void run() {

    System.out.println("Task" + taskId + "executed by"  
        + Thread.currentThread().  
            getName());

    try {  
        Thread.sleep(1000);

    } catch (InterruptedException e) {

        System.out.println(e);

    }  
}

public class ThreadPoolExample {

    public static void main(String[] args) {  
        ExecutorService executor = Executors.newFixedThread-  
            adPool(3); // Create thread  
                        pool of size 3

        for(int i=1; i<=5; i++) {

            executor.execute(new Task(i));

        } //submitting tasks to pool

        executor.shutdown();

        // shutting pool after  
        execution

23

## ⑥ Deadlock in Multithreading :->

Deadlock occurs when two threads are waiting on each other to release resources, causing an infinite wait.

Example ->

```
class Resource {
```

```
    synchronized void method1 (Resource r) {
```

```
        System.out.println (Thread.currentThread().  
                                getName() + " locked method1  
                                ");
```

```
        try {
```

```
            Thread.sleep(100);
```

```
        }
```

```
        catch (Exception e) {
```

```
        }
```

```
            r.method2();
```

```
        }
```

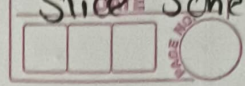
```
    }  
  
    synchronized void method2 {
```

```
        System.out.println (Thread.currentThread().  
                                getName() + "locked  
                                method2");
```

```
    }
```

```
}
```

\*\*\* Thread scheduling → Two types → ① Priority scheduling.  
② Time Slice Scheduling.



```
public class DeadlockExample {
```

```
    public static void main (String[] args) {
```

```
        Resource r1 = new Resource();
```

```
        Resource r2 = new Resource();
```

```
        Thread t1 = new Thread(() → r1.method1(r2));
```

```
        Thread t2 = new Thread(() → r2.method1(r1));
```

```
        t1.start();
```

```
        t2.start();
```

```
    }
```

```
}
```

⑦ Inter-thread Communication ⇒

Java provides `wait()`, `notify()`, and `notifyAll()`, for communication between threads.

Example →

```
class shared {
```

```
    synchronized void produce() {
```

```
        try {
```

```
            System.out.println("Producer waiting...");
```

```
            wait();
```

```
            System.out.println("Producer resumed...");
```

```
        }
```

DATE

```

    catch (InterruptedException e) {
        System.out.println(e);
    }
}

```

```

synchronized void consume() {
    System.out.println("Consumer processing...");
    notify(); // wake up waiting producer
}
}

```

```

public class InterThreadExample {
    public static void main (String[] args) {
        Shared obj = new Shared();

```

```

        Thread t1 = new Thread(1) -> obj.producer();
        Thread t2 = new Thread(1) -> obj.producer();

```

```

        t1.start();
        t2.start();
        try { Thread.sleep(1000); }
        catch (Exception e) {}
        t2.start();
    }
}

```

Conclusion →

- i) Multithreading improves performance and responsiveness.
- ii) Use Thread class or Runnable interface to create threads.
- iii) Synchronization prevents race conditions.
- iv) Thread pools optimize thread management.
- v) Be cautious of deadlocks and use inter-thread communication for co-ordination.

## Multithreading

i) Multiple threads within a single process

ii) Threads share the same memory space

iii) Speed is faster

iv) Communication is Easier (shared memory)

v) Tasks that share data is use case of Multithreading (e.g GUI + background tasks)

## Multiprocessing

i) Multiple independent processes.

ii) Processes have separate memory

iii) Speed is slower

iv) Communication is Harder (require IPC - inter process communication)

v) Heavy, isolated tasks is use case of Multiprocessing (e.g, video rendering)

## Example

viz Web server handling multiple users

## Example

viz Running multiple instances of a program

\*\*\*  
java.util.  
concurrent

Callable → (Callable is an interface in Java (like Runnable), used to define task for <sup>DATE</sup> thread)  
(But Runnable returns nothing while  
Callable can return a value and throw exception)

need to  
same  
as  
~~Runnable~~  
explicitly  
import

## Thread

i) A lightweight sub-unit of a process

ii) Thread share the same memory space

iii) Creation <sup>cost</sup> is low  
(faster to create)

## Process

i) An independent program in execution

ii) Processes have separate memory

iii) High Creation cost  
(slower to create)

util pack  
need to  
import  
explicitly

\*\*\*

## What is Runnable →

java.~~util~~  
lang

The Runnable interface is a functional interface in java used to represent a task that can be executed by a thread.

(It is part of java.lang package and has only one method.)

But we  
did not  
need to  
import it  
explicitly

```
public interface Runnable {  
    public abstract void run();  
}
```

It is  
automatically  
imported  
by java  
program

So if you wanted to define a task that runs in a thread, you implement the interface and write your logic inside the run() method

lang package  
automatically imported

In short →

Runnable = Better flexibility + clean code +  
Reusability + works with modern Java tools.

\* Why to use Runnable instead of Extending Thread?

① flexibility and Multiple Inheritance →

Java does not support multiple inheritance. If class extends Thread, it cannot extend any other class. But implementing Runnable allows the class to extend another class.

② Separation of concerns →

Your task logic and thread logic are separate. This makes your code easier to read and maintain.

③ can be reuse easily →

You can create one Runnable task and run it with multiple threads. No need to duplicate code.

Example :-

```
Runnable task = new MyTask();  
Thread t1 = new Thread(task);  
Thread t2 = new Thread(task);
```

④ Works great with Thread Pools (Executors) →

Modern Java uses ExecutorService (Thread.pool) for better performance.

These require tasks to be of type Runnable or Callable.

(So Runnable is perfect for advanced Multithreading).

(1)